LumaBot: Automated Desktop Lamp

Spring 2026 - MIE438 - Group 30

Demonstration Video

(A copy of the video is also located within this .zip file)

Hari Om Chada - 1010087068
Joshua Choi - 1010116786
Emmanuel Ko - 1010326096
Brooks Liu - 1009878027

1 - Motivation & Scope

Improper lighting during precision desk activities is known to reduce visibility, increase eye strain, and increase error rate during work. This is a common occurrence when the primary light source is static, such as a desk lamp or a headlamp, as the user's hands, tools, or head often obstruct the light path and cause shadows directly on the work zone. LumaBot aims to address this issue by automating the adjustment of lighting with minimal control needed from the user. This allows for concentration to stay on the task they are working on instead while also ensuring a good work environment is maintained.

Lumabot is a 5-DoF autonomous robotic lamp (arm) with an LED strip placed on its end-effector. Using computer vision (CV), it is able to track the user's hands, given they are wearing black or gray gloves, and adjust the joint angles so that the light is centered around their hands.

Originally, we wanted to have the light point under the hand, essentially tracking the area under the hand to minimize shadow. However, we realized that it was difficult to track objects under the hand, since then the computer vision model must generalize to be able to differentiate between the objects that the human is working on versus other objects in the camera frame. Therefore, we decided instead to do hand detection, and instead keep the light shining at the middle of the hand.

2 - Build Design Choices

This section outlines the choices of parts used to construct the arm of the lamp, why they were chosen, and challenges we faced. A wiring diagram and labelled image are shown in figure 1 and 2 for reference. Note that improvements that we can make are kept for section 4, and discussion of algorithms used to control the arm are explored in section 3.

2.1 - Controls

The two major pieces that were used in the project were the STM32H745I-DISCO microcontroller, and the Arducam Mini Module camera. The reason we chose this particular microcontroller was because it came with a screen, and we wanted to include a UI that the user could interact with, just like how they have the option to interact with a normal lamp. This gives the options to adjust the lamp manually (through sliders on the screen) or automate the process through the same menu. A few key specifications of the board that we had to keep in mind when choosing other aspects was the RAM (16MB), the 3.3V/5V outputs, and the 500mA current limit [1].

The inclusion of the camera was to aid the implementation of automated adjustments of the light. Images were taken by the Arducam camera, processed by the microcontroller, and turned into angles that were used to adjust the positioning of the arms. This particular camera was also chosen to fit within the limitations of the STM32 board, as a higher quality or higher frame rate camera would have increased the demand on the board for processing. The camera uses I2C for sensor configuration and SPI to transmit camera commands and the image data stream. The SPI speed operates at up to 8 MHz (at most that many bits per second), and the images sent obtained from the camera are at most 384KB [2], which are well within the 16MB external SDRAM or even the 1MB internal RAM.

Initially, we wanted to use the CAMERA-OV7670 to provide images to the microcontroller. However, this model did not have any onboard memory (no FIFO buffer). This meant that it sends unprocessed pixel data at its own clock rate unlike how the Arducam saves the entire picture to its own onboard chip. If the microcontroller is not continuously taking in data from the camera, it may miss some data, making the overall image unusable. Furthermore, the pin layouts conflicted with the ones on the board that were used by the screen, meaning that no proper connection can be established between them.

2.2 - Build

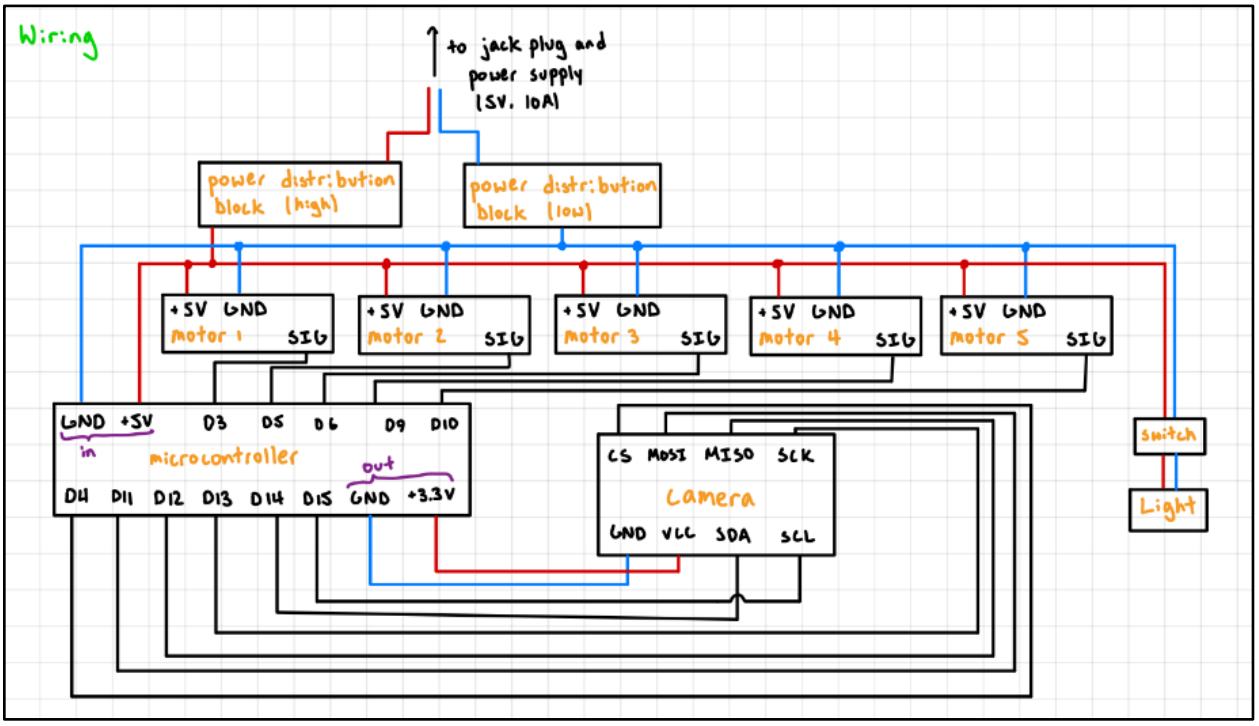

Figure 1. Wiring architecture diagram of LumaBot. Blue wires represent ground connections and red wires represent power connections.

We planned to use 3 MG996R motors and 2 MG90S mini servo motors to control the position of the robotic arm, as the CAD model we selected was designed for that particular selection. This would allow the weight of the arm to be fully supported by one motor even while turning. However, the final design features 5 MG996R servo motors instead. This was because when we were testing the MG90S mini servos, we found that they were continuous rotation models (meaning they take rotational speed as an input) instead of the expected positional rotation design (take an angle as input) that we expected [3]. For our design, positional rotation performs much better compared to continuous rotation, as we need to track precise angles of the arm in order to control motion; continuous rotation servos would make the tracking and control harder due to the increased uncertainty from setting/stopping based on a given speed.

The CAD model used for this project was found on GrabCAD [4], and can be found within the CAD directory. This specific model was chosen because of its simplicity,weight, and usage of only two motors that were within the course budget. A few modifications, such as cutting/filing the printed components, were made to create space in the joint connections to fit our new choice of larger motors (MG996R). As well, the claw of the original CAD was removed and replaced with a flat surface that we could attach the camera and light to. The light was a simple LED bar light that was connected to a toggle switch via wire soldered on both ends and connected to the power line.

Since each MG996R motor operates near 4.8-6V and pulls 500mA of current when running [5], the board is unable to power all 5 motors and the camera by itself. To address this, we used a 5V 10A power supply adaptor to power everything in parallel. The output of the supply was connected to a jack plug adaptor which separated it into separate ground and power rails. These rails were then connected to two power distribution blocks that connected to the individual electronics. We also had to ensure that the

current pulled (by the servos especially) could be handled by the wires, so 22AWG wires were used to connect them to the power distribution blocks, and 16AWG wires were used to connect the jack plug adapter to the power distribution blocks. This was necessary to prevent the wires from burning from overcurrent.

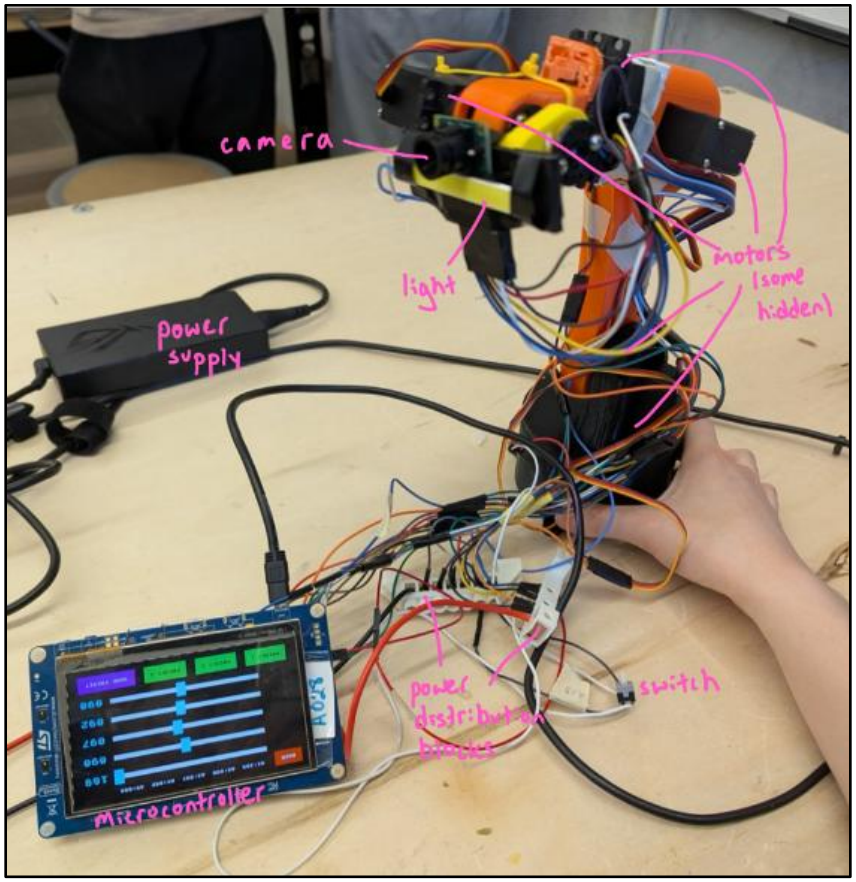

Figure 2. Labelled physical build.

3 - Software Pipeline and Control Design

As our system has a vision, motor actuation, and UI tasks, we used FreeRTOS to handle scheduling of these decoupled tasks [6], enabling them to run simultaneously as FreeRTOS rapidly switches between them (code in freertos.c). Naturally we have set the priority of the motor task as “Realtime” since it requires precise timing, vision task as “High” and UI as “Normal”. Additionally, DMA copies the pixels from the camera into RAM and the vision task is blocked until getting an interrupt when DMA finishes.

The end-to-end pipeline from image to actuation is as follows (when hand tracking). The camera captures the image frame and DMA transfers that information into RAM. The vision task is blocked until DMA finishes and the interrupt reenters the queue in RTOS. The frame is then processed in CV which returns target angles for every joint. The command is sent to the motor and since it has Realtime priority in RTOS, it acts immediately. Meanwhile, if applicable, the image and CV output get written to the UI. Also the user can change the mode of operation (discussed in 3.2) at any time to stop the hand tracking.

For tracking, we designed a simple control loop (code in arm_test.py) where we measure the pixel error between the centroid of the detected color's bounding box and the center of the image. We then use proportional control which returns the next target end-effector position.

3.1 - UI

This section is referring to code found in lcd_ui.c. Using the onboard touchscreen, the user can switch between modes of operation, outlined in 3.2. This is implemented like a finite state machine where mode switching is routed through a main menu screen. To avoid screen tearing we utilized ping-pong buffering, where the camera writes to one buffer while the UI reads from a second one, then they swap, this ensures that the UI never reads from a buffer that's actively being written. For this, we also utilized the VSYNC interrupt to trigger the buffer swap instead of constant CPU constantly polling. We also designed a custom UI engine that uses STM32's Chrom-ART (DMA2D) accelerator which offloads pixel rendering from the CPU.

3.2 - Servo Control and Modes of Operation

Lumabot's orientation can be set to our set neutral position at any time and it has 4 modes of operation, "Track", "Auto", "Manual", and "Debug". Track mode enables hand tracking, where LumaBot autonomously tracks hand movements, keeping the user's hand in the center of the frame. Furthermore, in track mode you can choose to record a trajectory and have LumaBot replay that movement immediately or save it to a preset for later use. Auto mode also does hand tracking but prioritizes keeping the end-effector at a set height, high up to keep the desk space consistently illuminated. Manual mode allows the user to directly input their desired angle for each joint using a slider and the given orientation can be saved as a preset as well. Debug mode displays a real-time, side-by-side split screen showing the raw camera feed next to the computer vision (CV) output. Specifically it shows bounding boxes which we used to calibrate color detection thresholds and tracking latency.

For both Track and Auto modes, an IK controller was used to solve for the end effector position. The high level controller would calculate the desired end effector position by incrementing the current end effector position towards the direction of the pixel error vector (error between center of image and hand detection coordinate), and then the IK controller tries to solve for that incremented desired position.

3.3 - Camera Inputs (angle_compute.c)

Initially, we trained Edge Impulse FOMO, which is an object detection algorithm, on the Google Hand Dataset of 4800 labelled data. We deemed that using Edge Impulse FOMO for hand detection is advantageous for deployment on STM32 microcontroller devices due to its efficiency and suitability for resource-constrained systems. Unlike traditional object detection models, FOMO eliminates bounding box regression and instead predicts object centroids on a low-resolution feature map, significantly reducing memory and computational requirements. This allows the model to fit within the limited RAM and flash available on STM32 microcontrollers while maintaining real-time inference performance. Additionally, FOMO performs well with relatively small datasets, such as the 4800 image Google Hand Dataset, due to its simplified prediction task and use of transfer learning. Its lightweight architecture also leads to lower power consumption, making it ideal for embedded, battery-powered applications requiring fast and reliable object detection.

Unfortunately, we found that deploying FOMO alongside our other system components, such as the UI and control logic on the STM32 led to tight RAM usage which led to inconsistent performance

across our software stack. Additionally, FOMO did not generalize well enough beyond the training set, specifically when encountering lighting variations and cluttered backgrounds. Though optimized, FOMO still struggled with inference speed on our hardware, making responses less stable overall.

Instead, we decided to use colour-based detection by identifying pixels that fall within a predefined colour threshold. We chose to wear blue gloves to ensure a more distinct and consistent colour profile, reducing interference from background objects and variations in skin tone or lighting. This approach simplifies the detection pipeline and improves robustness in controlled conditions. Additionally, it is significantly faster than using Edge Impulse FOMO, as it eliminates the need for neural network inference. As a result, it requires far less computational power and memory, making it especially well-suited for real-time performance on resource-constrained platforms such as STM32 microcontroller devices.

4 - Results and Reflection

4.1 - Results

We initially intended to design a 3-DoF robotic arm, but ended up going with 5 to allow it to cover a wider range of angles. This also affords the option to use less than 5-DoF for adjustments when we deem the extra ones unnecessary. This surpassed our original design in our proposal since with more degrees of freedom Lumabot has a larger field of view (FOV) which is one of our key design metrics.

For actuation accuracy, the system constantly moves towards the target end-effector position that results from our control loop and it does not oscillate or shift back and forth upon reaching the target position. This leads to motion smoothness and overall provides reliable illumination of the workspace.

In terms of latency, we found our design's speed was bottlenecked by image processing, which takes ~150-200ms. This comes from the time it takes for the camera to fully store, interrupt, and transmit the image over two parts, to the board, as well as the built in wait time to prevent any unintended results. This latency is moderate and seems to correspond to the FPS that we could see on the board during operation, but we did not notice any resulting responses being delayed even when comparing the system's responses when using a less compute-heavy processing pipeline. In other words, each image was being processed in time without affecting the processing of the next image, but the speed of processing (and hence, the target updating speed) might be a limiting factor in the ability to properly identify and track faster movements of the bounding box. The reason why we believe this is explained below.

One result that we thought was notable is the speed of rotation of the motors. While we are able to update the angles per tick that the motors turn (in angle_compute.c), setting them to high values (>5 degrees) makes it so that the motors will turn too fast when trying to reach the target value and accidentally overshoot so much that the bounding box is no longer within the camera's limits. We attributed this to be a limitation of the camera's capture rate because if we were able to capture images at a faster frequency, this would allow for us to record the overshoot and make adjustments.

4.2 - Future Improvements

As mentioned in section 4.1, the build is highly functional, but bottlenecked by the processing speed of the images. One solution could be to offload the processing to an external device through an ESP-32 module, for example, which affords the ability to use higher quality cameras that are able to capture at a much higher frequency. Once we are able to capture images at a higher FPS, a further improvement that could be made (and we would definitely try to implement knowing what we know now)

would be to try to implement a PI controller to see if we can reduce steady state error. More specifically, the angles per tick value can be dynamically reduced to prohibit unintended movement when we get close to centering the bounding box, and can be increased when the target value is moved away from center.

Considering what we learned about numbering systems and optimization, converting the floating point numbers used in computations within the vision and control loops to fixed point might be better for calculation speed. This is because we learned that MCUs are generally optimized for integer math so fixed-point operations are faster which allow for higher FPS and faster control responses. This also helps alleviate RAM requirements since fixed-point numbers are 16-bit while floats are at least 32-bit. However this number representation is less precise so we would need to test the speed vs accuracy tradeoff.

On a more holistic note, we also wanted to highlight some improvements that could be made to our work flow.

One improvement that could be made would be to test early and often when it comes to both software design and physical parts. For example, we wrote the code to process images from the camera and send a command to control the motors before we put together the full arm. While we did have simulations to test the functionality of this code, the physical test did not reflect the simulated results, causing us to need to debug over 500 lines of code. This process would have been a lot faster if we implemented the build earlier and were able to test on the physical system.

While we tried to be careful in our research about the compatibility of devices before we ordered them, it is still an area that can be worked on. The original camera not being compatible with our board is an example of this. We also ran into an issue with our board, with an overcurrent error occurring that we could not fix, which was likely due to a short within the processor. While we do not know the reason that the board broke, more research beforehand into precautions that we could take to ensure it does not short circuit may have been beneficial.

5 - References

[1] STMicroelectronics, “Discovery kits with STM32H745XI and STM32H750XB MCUs.” [Online]. Available: https://www.st.com/resource/en/user_manual/um2488-discovery-kits-with-stm32h745xi-and-stm32h750xb-mcus-stmicroelectronics.pdf

[2] Arduino, “OV2460 Camera Shield for Arduino.” [Online]. Available: https://dlscorp.com/wp-content/uploads/2018/04/ArduCAM-2.0-MP-Mini-Module-Data-Sheet.pdf

[3] electronicoscaldas, “MG90S Metal Gear Servo.” [Online]. Available: https://www.electronicoscaldas.com/datasheet/MG90S_Tower-Pro.pdf?srsltid=AfmBOorjMD4q2e39fPYyEZlAlm97F76uqJcjIys6EIZJDYVeD82lmwoy

[4] J. Liu, “5 DOF Robotic Arm,” GrabCAD. [Online]. Available: https://grabcad.com/library/5-dof-robotic-arm-6

[5]Handson Technology, “User Guide: MG996R Metal Gear Servo Motor.” [Online]. Available: https://www.handsontec.com/dataspecs/motor_fan/MG996R.pdf

[6] “What is FreeRTOS?,” FreeRTOS™. [Online]. Available: https://www.freertos.org/Why-FreeRTOS/What-is-FreeRTOS